Timer1 - notes complètes et mises au propre

ATmega328P • PB1 / OC1A • CTC • prescaler • PWM • rapport cyclique

Contenu conservé intégralement ; seule la présentation a été réorganisée et harmonisée.

1. PB1, OC1A et le lien avec Timer1

Le microcontrôleur a des broches, par exemple PB1. Une broche peut servir de sortie pour allumer ou éteindre une LED. Donc ici, ta LED est branchée sur PB1.

Maintenant, OC1A, ce n'est pas une autre broche physique. C'est une **fonction spéciale** que peut avoir la broche PB1.

Quand PB1 fonctionne normalement, c'est ton programme qui décide si elle vaut 0 ou 1.

Quand PB1 est utilisée comme OC1A, c'est le **Timer1** qui peut changer automatiquement son état.

Le 1 dans OC1A veut dire : **Timer1**.

Le A veut dire : **canal A du Timer1**.

OC veut dire **Output Compare**, qu'on peut traduire simplement par :

sortie commandée par une comparaison du timer.

Pour le moment, retiens seulement cette idée :

Timer1 compte des nombres.
Quand il arrive à une certaine valeur,
il peut agir directement sur OC1A.
Et comme OC1A est sur PB1,
il peut donc agir directement sur la LED.
Par exemple, très grossièrement :

Timer1 compte :

```
0
1
```

2
3
4
5 ← valeur choisie

À 5 :
PB1 change d'état

puis le timer recommence

2. Timer1 et le compteur TCNT1

Un **timer**, c’est simplement un compteur intégré dans le microcontrôleur.

Imagine une petite machine qui fait ça toute seule :

```
0
1
```

2
3
4
5
6
7
8
...
Elle avance grâce à l’horloge du microcontrôleur. Le processeur n’a pas besoin d’écrire chaque nombre lui-même.

Dans ton exercice, on utilise **Timer1**. Donc Timer1 possède une valeur qui augmente automatiquement.

Par exemple :

Timer1 = 0
Timer1 = 1
Timer1 = 2
Timer1 = 3
Timer1 = 4
...
Cette valeur est stockée dans un registre appelé :

```
TCNT1
```

> **Tu peux retenir :**

```
TCNT1 = la valeur actuelle du compteur Timer1
```

Par exemple, si tu regardes Timer1 à un instant donné, tu pourrais avoir :

```
TCNT1 = 153
```

Un peu plus tard :

```
TCNT1 = 154
```

puis :

```
TCNT1 = 155
```

Le truc intéressant, c’est qu’on peut dire à Timer1 :

« Quand tu arrives à une certaine valeur, fais quelque chose. »

Cette valeur cible sera stockée ailleurs. Plus tard, on verra OCR1A.

Pour l’instant, imagine juste :

Timer1 compte :

```
0
1
```

2
3
4
5 ← valeur cible atteinte !

À cet instant, le microcontrôleur détecte :

valeur du compteur = valeur cible

C’est ça qu’on appelle une **comparaison**.

Et c’est là que le nom Output Compare commence à avoir du sens.

Compare = comparer
Output = sortie

Donc OC1A est une sortie qui peut réagir lorsqu’il y a une comparaison réussie dans Timer1.

Visuellement :

Timer1 compte

```
      ↓
0 1 2 3 4 5
            ↑
  valeur cible
            ↓
  comparaison
            ↓
    OC1A
            ↓
     PB1
            ↓
     LED
```

Il faut aussi comprendre une chose importante : si Timer1 comptait à la vitesse maximale du microcontrôleur, il irait extrêmement vite.

Sur un microcontrôleur à 16 MHz, l’horloge fait environ :

16 000 000 cycles par seconde

Donc si le timer avançait à chaque cycle, il compterait des millions de fois par seconde.

Pour une LED qui doit clignoter lentement à **1 Hz**, c’est beaucoup trop rapide.

C’est pour ça qu’un peu plus tard on utilisera quelque chose appelé **prescaler**, qui sert simplement à ralentir le compteur.

Mais ne t’occupe pas encore des nombres.

À ce stade, retiens seulement ces quatre idées :

Timer1 = un compteur automatique.

`TCNT1` = la valeur actuelle de ce compteur.

On peut définir une valeur cible.

Quand le compteur atteint cette cible, Timer1 peut agir sur OC1A, donc sur PB1, donc sur la LED.

La suite logique, c’est de comprendre **ce qu’est un registre**, parce que TCNT1, TCCR1A, TCCR1B, etc. sont tous des registres. C’est vraiment la base avant de continuer.

3. Registres et bits : comment le logiciel configure le matériel

Un registre, c'est comme une petite zone de mémoire spéciale qui sert à **configurer ou lire le matériel** du microcontrôleur.

Par exemple, au lieu d'avoir une instruction du genre « active le Timer1 », tu modifies certains bits dans un registre. Le matériel lit ces bits et se comporte en conséquence.

Dans ton exercice, il y a plusieurs registres importants :

- `DDRB` : dit si PB1 est une entrée ou une sortie.
- `TCNT1` : contient la valeur actuelle du compteur Timer1.
- `TCCR1A` et `TCCR1B` : servent à configurer le fonctionnement de Timer1.
- `OCR1A` : contient la valeur cible avec laquelle Timer1 compare son compteur.

L'idée générale, c'est donc :

programme
↓
écrit dans les registres
↓
le matériel se configure
↓
Timer1 travaille tout seul

Par exemple :

```
DDRB |= (1 << DDB1);
```

ça veut dire en gros :

« Dans le registre DDRB, mets le bit correspondant à PB1 à 1. »

Et le matériel comprend :

« D'accord, PB1 devient une sortie. »

> **Le point important maintenant, c'est de comprendre ce qu'est un bit dans un registre, et surtout pourquoi on écrit des choses comme :**

```
(1 << DDB1)
```

C'est probablement la prochaine étape la plus utile avant de toucher à TCCR1A et TCCR1B.

4. TCCR1A et les bits COM1A1 / COM1A0

Alors on passe à TCCR1A, mais toujours sans formule.

`TCCR1A` est un registre qui sert à configurer une partie du **Timer1**.

Pour ton exercice, ce qui nous intéresse dedans, ce sont surtout deux bits :

```
COM1A1
COM1A0
```

Ces bits disent au Timer1 :

qu'est-ce que tu dois faire avec la sortie OC1A quand une comparaison se produit ?

Et comme OC1A correspond à PB1, ça revient à dire :

qu'est-ce que Timer1 doit faire sur PB1 ?

Les choix possibles sont du genre :

ne rien faire
mettre la sortie à 0
mettre la sortie à 1
inverser la sortie

Nous, pour faire clignoter la LED, ce qu'on veut c'est :

inverser la sortie

Donc :

0 devient 1
1 devient 0

Exemple :

PB1 = 0
comparaison
PB1 = 1
comparaison
PB1 = 0
comparaison
PB1 = 1

Pour demander ça au Timer1, on met :

```
COM1A1 = 0
COM1A0 = 1
```

Donc on peut écrire :

```
TCCR1A |= (1 << COM1A0);
```

Ça veut dire :

mets le bit COM1A0 à 1.

Et comme `COM1A1` reste à 0, la combinaison devient :

```
COM1A1 COM1A0
  0      1
```

Ce mode signifie :

Toggle OC1A on Compare Match

En français :

inverse l'état de OC1A à chaque comparaison.

Donc à ce moment-là, le lien devient :

Timer1 fait une comparaison

```
       ↓
COM1A1/COM1A0 disent quoi faire
       ↓
```

OC1A change d'état

```
       ↓
```

PB1 change d'état

```
       ↓
```

LED change d'état

Le commentaire que tu pourrais mettre dans ton code serait par exemple :

```
TCCR1A |= (1 << COM1A0); // Configure OC1A pour inverser PB1 à chaque comparaison du Timer1
```

`COM1A0`, c'est simplement **le nom d'un bit dans le registre `TCCR1A`.**

Décomposons le nom :

- COM = **Compare Output Mode**
- 1 = ça concerne **Timer1**
- A = ça concerne la sortie/canal **A**
- 0 = c'est le **bit numéro 0 de ce réglage**

Il y a en fait deux bits qui vont ensemble :

```
COM1A1
COM1A0
```

À eux deux, ils choisissent **ce que Timer1 doit faire sur OC1A** quand une comparaison arrive.

Donc :

```
COM1A1 COM1A0
  0      0   → ne rien faire
  0      1   → inverser OC1A
```

...

Dans ton exercice, on veut inverser la LED, donc on veut :

```
COM1A1 = 0
COM1A0 = 1
```

Et comme `COM1A0` est juste un bit dans `TCCR1A`, on peut écrire :

```
TCCR1A |= (1 << COM1A0);
```

Ça veut juste dire :

mets le bit COM1A0 du registre TCCR1A à 1.

> **Le point important : COM1A0 n'est pas une broche, ni un timer, ni un registre. C'est juste un bit de configuration.**

Et OC1A, lui, c'est la sortie spéciale liée au Timer1. Donc ne confonds pas :

```
COM1A0 → bit de configuration
OC1A   → sortie spéciale du Timer1
PB1    → broche physique
```

Donc COM1A1 et COM1A0 vont ensemble pour choisir parmi plusieurs comportements.

Imagine un petit interrupteur à 4 positions :

```
COM1A1 COM1A0
  0      0   → comportement 1
  0      1   → comportement 2
  1      0   → comportement 3
  1      1   → comportement 4
```

Dans notre cas, on veut le comportement :

inverser OC1A à chaque comparaison.

Et pour ça, la combinaison est :

```
COM1A1 = 0
COM1A0 = 1
```

Donc on obtient :

```
01
```

Tu peux voir ça comme un code binaire qui sélectionne une option.

Et attention à un petit détail : quand on écrit

```
TCCR1A |= (1 << COM1A0);
```

on met juste COM1A0 à 1.

On suppose donc que COM1A1 est déjà à 0, ce qui est normalement le cas au démarrage.

Si tu veux être très clair dans un exercice scolaire, tu peux même penser :

```
COM1A1 = 0
COM1A0 = 1
```

donc :

mode choisi = toggle / inversion
La logique à retenir est juste :

2 bits = 4 choix possibles
C’est pour ça qu’il y en a deux.

5. Retrouver les informations dans le datasheet

Oui — dans **ton datasheet**, l'info est très précisément à la **page 108**, section **15.11.1 — TCCR1A – Timer/Counter1 Control Register A**.

En haut de la page, tu as ce dessin du registre :

Bit	7	6	5	4	3	2	1	0
	COM1A1	COM1A0	COM1B1	COM1B0	–	–	WGM11	WGM10

Donc ici tu apprends déjà quelque chose d'important :

```
COM1A1 = bit 7 de TCCR1A
COM1A0 = bit 6 de TCCR1A
```

Petite correction importante par rapport à ce qu'on disait avant : le 0 dans COM1A0 **ne veut pas dire que c'est le bit 0 du registre**. COM1A1 et COM1A0 sont simplement les deux bits d'un même réglage. Dans TCCR1A, ils sont physiquement aux positions 7 et 6.

Ensuite, juste en dessous, le datasheet écrit :

```
Bit 7:6 – COM1A1:0: Compare Output Mode for Channel A
```

Ça veut dire que les bits 7 et 6 servent ensemble à choisir le comportement de la sortie **OC1A**.

Et surtout, regarde la **Table 15-2 — Compare Output Mode, non-PWM**. C'est celle qui nous intéressera parce qu'on va utiliser le Timer en mode CTC.

Tu y trouves :

COM1A1	COM1A0	Résultat
0	0	fonctionnement normal de la broche
0	1	Toggle OC1A on compare match
1	0	mettre OC1A à 0 lors de la comparaison
1	1	mettre OC1A à 1 lors de la comparaison

Donc notre choix :

```
COM1A1 = 0
COM1A0 = 1
```

vient **directement de cette table**.

Et la phrase :

Toggle OC1A on compare match
signifie simplement :

inverse OC1A lorsqu'une comparaison arrive.

Donc :

```
0 → 1
1 → 0
```

Comment retrouver ça toi-même dans un datasheet

Quand un exercice te parle de TCCR1A, ne lis surtout pas les 300 pages dans l'ordre 😄. Fais Ctrl + F et cherche :

```
TCCR1A
```

Tu vas arriver à la section du registre. Ensuite tu regardes :

1. le dessin du registre pour savoir o**ù sont les bits** ;
2. la description en dessous pour savoir à **quoi ils servent** ;
3. les tableaux pour savoir q**uelle combinaison de 0 et de 1 choisir.**
4. Pour notre exercice, tu peux donc noter dans tes notes :

Datasheet p.108

```
TCCR1A
COM1A1:COM1A0 = 01
→ Toggle OC1A on Compare Match
```

Et il y a même une autre phrase très importante sur cette page : le datasheet précise que lorsque OC1A prend le contrôle de la broche, l**e bit DDR correspondant doit quand même être configuré en sortie.**

C'est exactement pourquoi on doit quand même faire :

```
DDRB |= (1 << PB1);
```

même si après on n'utilise pas PORTB.

La prochaine chose qu'on devrait chercher ensemble dans t**on datasheet,** c'est où il est écrit que P**B1 correspond à OC1A.** Comme ça tu verras que ça aussi, on ne l'invente pas.

6. OCR1A et le Compare Match

OCR1A, c'est un r**egistre qui contient la valeur cible** du Timer1.

Tu peux le voir comme ça :

```
TCNT1 = compteur actuel
OCR1A = valeur à atteindre
```

Exemple simple :

```
TCNT1 compte :
0
1
```

2
3
4
5

et si :

```
OCR1A = 5
```

alors quand TCNT1 arrive à 5, le Timer1 voit :

```
TCNT1 == OCR1A
```

et ça crée ce qu'on appelle un c**ompare match.**

Dans notre cas, ce compare match va servir à faire changer l'état de OC1A, donc de PB1, parce qu'on a configuré :

```
COM1A1 = 0
COM1A0 = 1
```

Donc l'enchaînement est :

```
TCNT1 compte
  ↓
```

arrive à OCR1A

```
  ↓
```

compare match

```
  ↓
```

OC1A change d'état

```
  ↓
```

PB1 change d'état

```
  ↓
```

LED change d'état

Exemple ultra simple :

```
OCR1A = 5;
```

ça voudrait dire :

quand le compteur atteint 5, déclenche la comparaison.

Mais pour ton exercice, 5 serait beaucoup trop petit, donc la LED changerait beaucoup trop vite.

> **Pour l’instant, retiens juste :**

```
OCR1A = la valeur cible du Timer1
```

et `TCNT1` est :

la valeur actuelle du compteur

La prochaine étape, c’est de comprendre p**ourquoi le compteur repart à zéro après la comparaison,** parce que ça nous amène au mode CTC.

7. Mode CTC

Alors, le C**TC** c'est un mode de fonctionnement du Timer1.

CTC veut dire :

Clear Timer on Compare Match

En français :

remettre le compteur à zéro quand il atteint la valeur de comparaison.

Imagine que :

```
OCR1A = 5
```

Le compteur fait :

```
TCNT1 :
0
1
```

2
3
4
5 ← comparaison avec OCR1A

À ce moment-là, en mode CTC :

```
TCNT1 revient à 0
```

Puis il recommence :

```
0
1
```

2
3
4
5

```
0
1
```

2
3
4
5
...

Donc ça crée un cycle régulier.

Et comme on a configuré OC1A pour changer d'état à chaque comparaison, ça donne :

compteur atteint OCR1A

```
        ↓
```

OC1A change d’état

```
        ↓
```

compteur repart à 0

```
        ↓
```

il recompte

```
        ↓
```

OC1A change encore d’état

Donc la LED fait :

OFF

```
↓
```

comparaison

ON

```
↓
```

comparaison

OFF

```
↓
```

comparaison

ON

C’est exactement ce qu’on veut.

Le mode CTC est activé avec un bit qui s’appelle :

```
WGM12
```

et ce bit se trouve dans :

```
TCCR1B
```

Pour l’instant retiens juste :

```
CTC = Timer compte jusqu’à OCR1A puis repart à 0
```

et :

```
WGM12 = bit qui permet d’activer ce mode
```

8. TCCR1B

`TCCR1B`, c'est un autre registre de configuration du T**imer1.**

Dans ce registre, il y a plusieurs bits. Pour nous, il y en a deux types importants :

`WGM12` → choisit le m**ode CTC**

`CS12`, `CS11`, `CS10` → choisissent la v**itesse du timer**

Commençons juste par WGM12.

WGM veut dire W**aveform Generation Mode.** En gros, ces bits servent à choisir c**omment le timer fonctionne.**

Pour mettre Timer1 en mode CTC, on doit mettre :

```
WGM12 = 1
```

et certains autres bits WGM à 0.

Donc on peut écrire :

```
TCCR1B = (1 << WGM12);
```

À ce moment-là, on dit essentiellement :

Timer1 fonctionne en mode CTC.

Ce qui donne le comportement :

```
TCNT1 compte
0
1
```

2
3
...

```
OCR1A
↓
```

comparaison

```
↓
```

retour à 0

Mais attention : si on écrit seulement :

```
TCCR1B = (1 << WGM12);
```

le timer n'a encore p**as de vitesse choisie.** Il ne va donc pas réellement compter comme on le souhaite.

C'est là qu'arrivent :

```
CS12
CS11
CS10
```

Eux servent à choisir la vitesse du Timer1.

Donc pense à `TCCR1B` comme à un registre qui répond à deux questions :

1. Comment le timer fonctionne ?
   → `WGM12 → CTC`

2. À quelle vitesse il compte ?
   → `CS12 / CS11 / CS10`

La prochaine notion à comprendre, c'est donc le p**rescaler,** c'est-à-dire comment on ralentit le Timer1

9. TCCR1A et TCCR1B - détail complet des bits

Oui. Je te fais les d**eux registres complets,** bit par bit, en expliquant à quoi ils servent même si on ne les utilise pas dans ton exercice.

Le plus important à comprendre avant : TCCR1A et TCCR1B sont deux registres qui servent à c**onfigurer Timer1.** Ils ne contiennent pas la valeur du compteur lui-même : ça, c'est TCNT1.

1. TCCR1A

Il ressemble à ça :

```
Bit     7     6     5     4     3     2     1     0
        COM1A1  COM1A0  COM1B1  COM1B0     –        –     WGM11
WGM10
```

Bits 7 et 6 : COM1A1 et COM1A0

COM = C**ompare Output Mode**

Ils contrôlent le comportement de la sortie O**C1A.**

Dans ton cas :

```
OC1A → PB1
```

Donc ces deux bits permettent de dire :

Quand Timer1 fait une comparaison, qu'est-ce qu'on fait sur OC1A ?

En mode non-PWM, par exemple CTC :

```
COM1A1 COM1A0
```

```
  0      0    → OC1A n'est pas contrôlée par Timer1
  0      1    → OC1A change d'état : 0→1 ou 1→0
  1      0    → OC1A est forcée à 0
  1      1    → OC1A est forcée à 1
```

Pour ton exercice :

```
0 1
```

parce qu'on veut :

```
OFF → ON → OFF → ON → ...
```

Donc :

```
TCCR1A = (1 << COM1A0);
```

2. Bits 5 et 4 : COM1B1 et COM1B0

Exactement le même principe, sauf qu'ils contrôlent :

OC1B
au lieu de :

OC1A
Timer1 possède en fait d**eux canaux de comparaison** :
Timer1

```
├── canal A → OC1A
└── canal B → OC1B
```

Donc :

```
COM1A1 / COM1A0 → contrôlent OC1A
COM1B1 / COM1B0 → contrôlent OC1B
```

Sur un ATmega328P :

```
OC1A → PB1
OC1B → PB2
```

Ça veut dire que Timer1 pourrait par exemple commander automatiquement d**eux sorties différentes.**

Dans ton exercice, ta LED est sur PB1, donc on utilise seulement le canal A.

3. Bits 3 et 2 : réservés

Ils apparaissent généralement comme :

-
-
Ça veut dire :

Ces bits ne sont pas utilisés.

Ils sont réservés par le constructeur.

En général, tu ne dois pas essayer de les utiliser ou de leur donner une signification.

Tu peux simplement les considérer comme :

```
bit 3 → rien
bit 2 → rien
```

4. Bits 1 et 0 : WGM11 et WGM10

WGM signifie :

Waveform Generation Mode

Le nom paraît compliqué, mais pense simplement :

Ces bits servent à choisir la manière dont Timer1 compte et fonctionne.

Par exemple Timer1 peut fonctionner de différentes façons :

mode normal
CTC
PWM
Fast PWM
Phase Correct PWM
...
Mais deux bits ne suffisent pas pour choisir tous ces modes.

C’est pour ça qu’on a :

```
TCCR1A :
WGM11
WGM10
```

et aussi dans TCCR1B :

```
WGM13
WGM12
```

Les quatre bits fonctionnent ensemble :

```
WGM13 WGM12 WGM11 WGM10
```

Ils forment un nombre à 4 bits qui choisit le mode.

Par exemple pour le C**TC** que nous allons utiliser :

```
WGM13 = 0
WGM12 = 1
WGM11 = 0
WGM10 = 0
```

donc :

```
0100
```

Résumé de TCCR1A

```
TCCR1A

COM1A1 COM1A0
   ↓
```

Contrôle la sortie OC1A / PB1

COM1B1 COM1B0

```
   ↓
```

Contrôle la sortie OC1B / PB2

bits 3 et 2

```
   ↓
```

réservés

```
WGM11 WGM10
   ↓
```

participent au choix du mode Timer1

Maintenant TCCR1B

Le registre ressemble à ça :

Bit 7 6 5 4 3 2 1 0
ICNC1 ICES1 - WGM13 WGM12 CS12 CS11 CS10

Il y a trois grandes catégories :

ICNC1 / ICES1

```
→ Input Capture
```

```
WGM13 / WGM12
→ mode du Timer1
```

```
CS12 / CS11 / CS10
→ vitesse du Timer1
```

5. Bit 7 : ICNC1

ICNC signifie :

Input Capture Noise Canceler

C’est une fonction qui sert avec l’I**nput Capture.**

Timer1 peut surveiller une broche appelée :

ICP1
Imaginons que tu branches un capteur dessus.

Lorsqu’un signal arrive :

```
_____|‾‾‾‾‾
     ↑
```

Timer1 peut enregistrer exactement :

à quelle valeur se trouvait le compteur à cet instant.

Par exemple :

```
TCNT1 = 8472
```

signal arrive

```
→ Timer1 sauvegarde 8472
```

Ça permet de mesurer précisément des temps, périodes, fréquences, etc.

Mais les signaux électriques peuvent avoir du bruit :

```
_____|_|_|‾‾‾
```

Le bit :

ICNC1 = 1
active un petit filtre pour éviter que Timer1 prenne un parasite électrique pour un vrai changement de signal.

Donc très simplement :

ICNC1 = 0

```
→ pas de filtre
```

ICNC1 = 1

```
→ filtre anti-bruit activé
```

On ne l'utilise pas dans ton exercice.

6. Bit 6 : ICES1

ICES signifie :

Input Capture Edge Select

Il sert aussi avec l'Input Capture.

Un signal numérique peut changer :

```
0 → 1
```

ça s'appelle un f**ront montant** :

```
_____|
     |_____
```

ou :

```
1 → 0
```

un f**ront descendant** :

```
_____|

     |_____
```

ICES1 choisit lequel Timer1 doit détecter.
ICES1 = 0

```
→ front descendant
```

ICES1 = 1

```
→ front montant
```

Par exemple :

ICES1 = 1
voudrait dire :

Capture la valeur du compteur lorsqu'un signal passe de 0 à 1.

Encore une fois, pas utile pour ta LED.

7. Bit 5 : réservé

Même chose que tout à l'heure :

bit 5 = -
On ne l'utilise pas.

8. Bit 4 : WGM13

C'est un des quatre bits qui choisissent le mode de Timer1.

Il fonctionne avec :

```
WGM12
WGM11
WGM10
```

Donc :

```
WGM13 ┐
WGM12 │
WGM11 ├→ choix du mode Timer1
WGM10 ┘
```

Tout seul, WGM13 ne veut pas vraiment dire quelque chose.

Il faut regarder les quatre ensemble.

9. Bit 3 : WGM12

Même principe.

Il fait partie du choix du mode :

```
WGM13 WGM12 WGM11 WGM10
```

Dans n**otre exercice,** il sera particulièrement important parce que le mode CTC que nous voulons correspond à :

```
0 1 0 0
 ↑
WGM12 = 1
```

Donc on aura :

```
(1 << WGM12)
```

10. Bits 2, 1 et 0 : CS12, CS11, CS10

Ceux-là sont très importants.

CS signifie :

Clock Select

Ils choisissent l**a source et la vitesse de l'horloge de Timer1.**

En gros :

À quelle vitesse Timer1 doit-il compter ?

Les trois bits ensemble donnent plusieurs possibilités :

```
CS12 CS11 CS10
```

```
0    0    0   → Timer arrêté
0    0    1   → horloge normale
0    1    0   → horloge / 8
0    1    1   → horloge / 64
1    0    0   → horloge / 256
1    0    1   → horloge / 1024
```

Les deux dernières combinaisons servent à utiliser une horloge externe, on y reviendra si besoin.

Le /8, /64, /256, etc., c'est ce qu'on appelle le p**rescaler.**

Par exemple, si le microcontrôleur travaille à :

16 000 000 Hz
et que tu choisis :

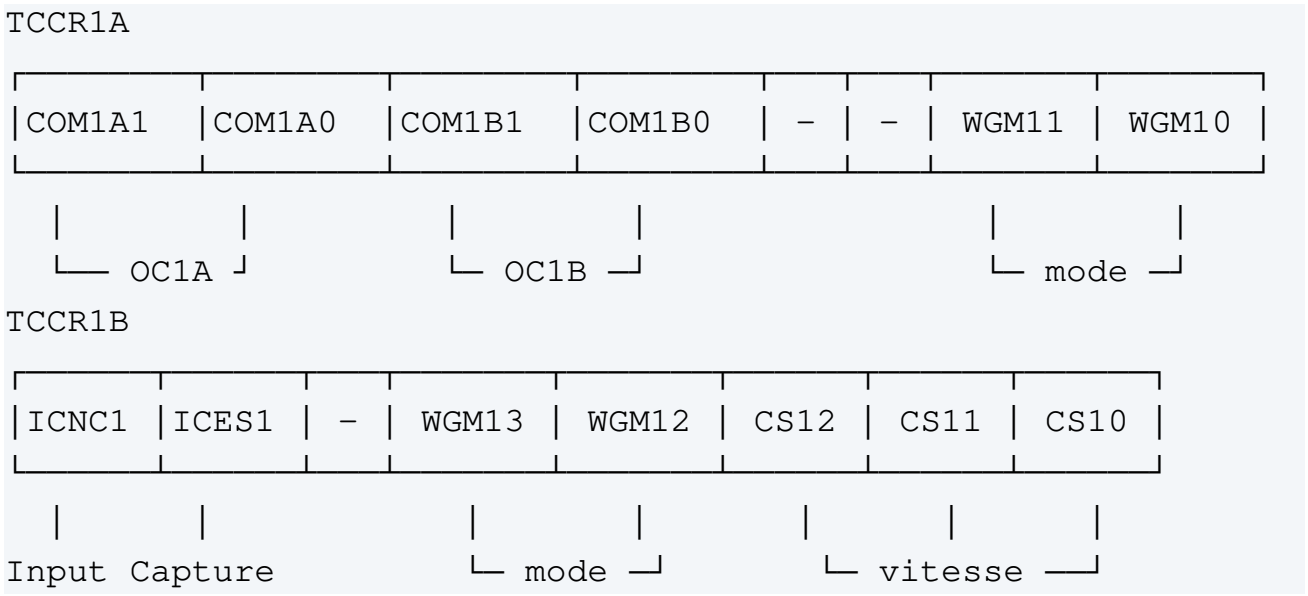
/256
Timer1 reçoit une horloge beaucoup plus lente.

C'est justement ce qu'on utilisera pour que ta LED clignote lentement.

Mais on expliquera ça séparément parce que c'est une notion importante.

Vue complète des deux registres

Tu peux garder ça comme fiche :



Et mentalement, tu peux réduire tout ça à :

```
TCCR1A
→ Que faire avec OC1A / OC1B ?
→ une partie du mode
```

```
TCCR1B
→ fonctions Input Capture
→ autre partie du mode
→ vitesse du Timer1
```

Pour t**on exercice,** parmi tous ces bits, on finira principalement par utiliser :

```
COM1A0 → faire basculer PB1
WGM12  → mode CTC
CSxx   → choisir la vitesse du Timer1
```

La prochaine étape importante est le p**rescaler,** parce que sans lui on ne peut pas comprendre comment obtenir exactement 1 Hz.

10. Prescaler : ralentir l’horloge du Timer1

On passe au p**rescaler,** tranquillement.

Le problème de base, c’est que le microcontrôleur va très vite. Si ton ATmega tourne à 16 MHz, ça veut dire que son horloge fait environ :

16 000 000 impulsions par seconde
Si Timer1 comptait directement avec cette horloge, il compterait beaucoup trop vite pour faire clignoter une LED lentement.

Le prescaler sert donc à r**alentir l’horloge envoyée au Timer1.**

Imagine :

Horloge CPU : tic tic tic tic tic tic tic tic ...
Avec un prescaler /8, Timer1 ne compte qu’une fois toutes les 8 impulsions :

CPU : tic tic tic tic tic tic tic tic
Timer1 : +1
Donc /8 veut dire :

Timer1 compte 8 fois moins vite que l’horloge du processeur.

Même idée pour :

/64
/256
/1024
Plus le nombre est grand, plus Timer1 compte lentement.

Les bits qui choisissent ça sont dans `TCCR1B` :

```
CS12 CS11 CS10
```

Ils forment ensemble un choix :

```
CS12 CS11 CS10

0    0    0   → Timer arrêté
0    0    1   → pas de division
0    1    0   → /8
0    1    1   → /64
1    0    0   → /256
1    0    1   → /1024
```

Pour notre exercice, on peut choisir /256.

Donc :

```
CS12 = 1
CS11 = 0
CS10 = 0
```

Et en code :

```
TCCR1B = (1 << WGM12) | (1 << CS12);
```

Cette ligne fait deux choses en même temps :

```
WGM12 = 1 → mode CTC
CS12  = 1 → prescaler /256
```

Donc Timer1 fonctionne en CTC et compte plus lentement.

Avec 16 MHz et /256, Timer1 reçoit en gros :

16 000 000 / 256
= 62 500
Donc Timer1 fait environ :

62 500 comptages par seconde
Ça devient beaucoup plus pratique pour fabriquer une période de 1 seconde.

> **Pour l’instant, retiens juste :**

prescaler = ralentisseur du Timer1
et :

```
CS12 CS11 CS10 = choix du ralentissement
```

11. Calcul de la valeur OCR1A pour obtenir 1 Hz en CTC

D’accord. On part de ce qu’on a déjà :
Horloge CPU = 16 000 000 Hz
Prescaler = 256

Donc Timer1 compte :
16 000 000 / 256 = 62 500 fois par seconde
Ça veut dire qu’en 1 **seconde,** le compteur avance de 62 500 pas.

Mais attention : notre LED change d’état à chaque comparaison.

Donc pour faire :

```
OFF → ON → OFF
```

il faut d**eux comparaisons.**
Si on veut une fréquence de 1 Hz, un cycle complet doit durer 1 seconde.

Donc chaque changement d’état doit arriver toutes les :

0,5 seconde
Puisque Timer1 fait 62 500 comptages par seconde, en une demi-seconde il fait :

62 500 / 2 = 31 250 comptages
Donc on veut une comparaison tous les 31 250 comptages.

On pourrait penser :

```
OCR1A = 31250;
```

mais le compteur commence à 0.

Il compte donc :

```
0
1
```

2
3
...
31249
Ça représente déjà 3**1 250 valeurs.**

Donc on met :

```
OCR1A = 31249;
```

Et le fonctionnement devient :

```
TCNT1 compte jusqu'à 31249
        ↓
```

comparaison

```
        ↓
```

PB1 change d'état

```
        ↓
TCNT1 repart à 0
```

Puis 0,5 seconde plus tard :

nouvelle comparaison

```
        ↓
```

PB1 change encore d'état

Donc :

```
0,0 s  → LED OFF
0,5 s  → LED ON
1,0 s  → LED OFF
1,5 s  → LED ON
2,0 s  → LED OFF
```

Donc une période complète dure bien :

1 seconde

et donc :

fréquence = 1 Hz

La valeur importante à retenir ici est donc :

```
OCR1A = 31249;
```

Et le commentaire pourrait être :

OCR1A = 31249; // Compare toutes les 0,5 s avec un prescaler de 256, afin d'obtenir un signal de 1 Hz

12. Première application : 1 Hz avec un rapport cyclique de 10 %

À 1 **Hz,** une période complète dure 1 **seconde.** Avec un rapport cyclique de 1**0 %,** la LED doit être :

```
allumée pendant 10 % de 1 s  → 0,1 s
éteinte pendant 90 % de 1 s  → 0,9 s
```

Donc on veut quelque chose comme :

ON : ██
OFF : ██████████████████

<------ 1 s ------>

Ici, Timer1 doit générer directement ce signal sur OC1A, donc sur PB1.

Pour faire ça proprement, on peut utiliser Timer1 en F**ast PWM,** avec ICR1 comme valeur maximale du compteur.

L'idée est :

```
TCNT1 compte de 0 jusqu'à ICR1
```

0 ---------- ICR1
|---- ON ----|------ OFF -------|

```
     ↑
   OCR1A
```

ICR1 va donc fixer la d**urée totale de la période,** et OCR1A va fixer c**ombien de temps la LED reste allumée.**
Avec un microcontrôleur à 1**6 MHz** et un prescaler de 2**56** :

16 000 000 / 256 = 62 500
Timer1 avance donc de 6**2 500 pas par seconde.**

Comme on veut une période de 1 **seconde,** il doit compter 62 500 valeurs :

```
ICR1 = 62499;
```

Pourquoi 62499 ? Parce qu'il compte de 0 à 62499, soit :

62 500 valeurs
Pour garder la LED allumée pendant 10 % :

10 % de 62 500 = 6 250
donc :

```
OCR1A = 6250;
```

Le compteur fait alors, en simplifiant :

```
TCNT1
```

0 6250 62499
|---------|-------------------------------|
ON OFF

10 % 90 %

Maintenant les registres.

Pour que OC1A fonctionne en PWM n**on-inversé,** on utilise :

```
COM1A1 = 1
COM1A0 = 0
```

Ça signifie en gros :

mettre OC1A à 1 au début de la période, puis la mettre à 0 lorsqu'on atteint OCR1A.

Donc :

```
TCCR1A = (1 << COM1A1) | ...;
```

Ensuite, le mode qu'on veut est F**ast PWM, TOP = ICR1,** qui est le mode 14 de Timer1.

Pour ce mode, les bits WGM sont :

```
WGM13 WGM12 WGM11 WGM10
 1     1     1     0
```

Comme WGM11 se trouve dans TCCR1A :

```
TCCR1A = (1 << COM1A1) | (1 << WGM11);
```

Et WGM13 + WGM12 sont dans TCCR1B :

```
TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS12);
```

CS12 = 1 avec CS11 = 0 et CS10 = 0 donne le prescaler /256.

13. Comprendre le rapport cyclique

Un r**apport cyclique,** c’est simplement le pourcentage du temps pendant lequel le signal reste à 1 pendant une période complète.

Imagine une période de 1 seconde.

Avec un rapport cyclique de 50 % :

```
ON  : ████
OFF : ████
```

1 seconde

La LED est allumée la moitié du temps, donc :

ON = 0,5 s
OFF = 0,5 s

Avec un rapport cyclique de 10 % :

ON : ▉

```
OFF : ████████
```

1 seconde

La LED est allumée seulement 10 % du temps.

Donc si la fréquence est 1 Hz, une période dure 1 seconde, et :

10 % de 1 seconde = 0,1 seconde

Donc :

LED allumée = 0,1 s
LED éteinte = 0,9 s

Puis ça recommence :

ON 0,1 s
OFF 0,9 s
ON 0,1 s
OFF 0,9 s
...

Le rapport cyclique, c’est donc simplement :

temps où la sortie est à 1
---
durée totale de la période

Pour ton exercice :

fréquence = 1 Hz

```
→ période = 1 seconde
```

rapport cyclique = 10 %

```
→ LED allumée 0,1 seconde
→ LED éteinte 0,9 seconde
```

14. PWM et modes de fonctionnement du Timer1

Oui. Là on arrive à un gros morceau du Timer1, donc je vais d'abord t'expliquer c**e qu'est le PWM,** puis je te montre les différents modes du Timer1 sans encore rentrer dans les calculs.

Le P**WM** veut dire P**ulse Width Modulation,** en français « modulation de largeur d'impulsion ».

L'idée est très simple : on fait un signal qui alterne entre 1 et 0, mais on choisit c**ombien de temps il reste à 1.**

Par exemple, avec une période de 1 seconde :

Rapport cyclique 50 %

```
HIGH : ▉▉▉▉▉
LOW  : ▉▉▉▉▉
```

1 seconde

Ici la sortie est à 1 pendant la moitié du temps.

Avec 10 % :

HIGH : ▉

```
LOW  : ▉▉▉▉▉▉▉▉▉
```

1 seconde

Avec 80 % :

```
HIGH : ▉▉▉▉▉▉▉▉
```

LOW : ▉▉

1 seconde

Donc le PWM sert à contrôler la proportion de temps passée à 1.

C'est très utilisé pour faire varier une luminosité de LED, la vitesse d'un moteur, créer certains signaux, etc.

Comment le Timer crée du PWM

Imagine que Timer1 compte :

0 1 2 3 4 5 6 7 8 9

On décide que :

TOP = 9

TOP veut simplement dire :
la valeur maximale atteinte par le compteur.

On choisit aussi une valeur de comparaison, par exemple :

```
OCR1A = 3
```

Le Timer peut alors faire :

compteur :
0 1 2 3 4 5 6 7 8 9

sortie :

```
1 1 1 0 0 0 0 0 0 0
```

Puis il recommence.

Donc TOP contrôle principalement l**a durée totale de la période,** tandis que OCR1A contrôle à **quel moment la sortie change d'état.**

C'est ça qui va nous permettre de faire tes 10 %.

Les différents modes de Timer1

Timer1 possède plusieurs modes parce qu'on peut vouloir qu'il compte de différentes manières.

Les quatre bits :

```
WGM13 WGM12 WGM11 WGM10
```

choisissent le mode.

WGM = W**aveform Generation Mode.**

Il y a 1**6 combinaisons possibles,** de 0000 à 1111.

Mais tu n'as pas besoin de les apprendre par cœur. Il faut surtout comprendre les grandes familles.

Mode 0 — Normal

WGM = 0000
Dans ce mode, Timer1 compte simplement :

```
0
```

```
1
```

2
3
...
65534
65535

```
0
1
```

...
Timer1 est sur 16 bits, donc sa valeur maximale naturelle est :

65535
Après, il déborde et revient à 0.

On appelle ça un o**verflow.**

Ce mode est utile quand tu veux simplement mesurer du temps ou déclencher quelque chose lors d’un débordement.

Mode CTC

On a déjà utilisé cette idée dans ton exercice précédent.

CTC = C**lear Timer on Compare Match.**

Le Timer compte :

```
0
1
```

2
3
...

```
OCR1A
```

puis :

```
→ retour à 0
```

Donc si :

```
OCR1A = 5
```

tu as :

0 1 2 3 4 5
0 1 2 3 4 5

0 1 2 3 4 5
C’est très pratique pour créer une fréquence précise.

Il existe deux versions principales dans Timer1.

Mode 4 — CTC avec OCR1A

WGM = 0100
Ici :

```
TOP = OCR1A
```

Donc OCR1A définit jusqu’où le compteur monte.

C’est ce qu’on utilisait pour ton premier exercice à 1 Hz.

Mode 12 — CTC avec ICR1

WGM = 1100
Même principe, mais cette fois :

```
TOP = ICR1
```

Donc c’est ICR1 qui définit la valeur maximale.

Ça permet de garder OCR1A disponible pour faire autre chose.

Maintenant les modes PWM

Il y a plusieurs sortes de PWM.

Les trois importantes à comprendre sont :

Fast PWM
Phase Correct PWM
Phase and Frequency Correct PWM

Fast PWM

C’est celui qui nous intéresse le plus pour ton exercice.

Le compteur fait :

```
0
```

```
1
```

2
3
4
...
TOP
puis il revient directement à :

```
0
```

Donc :

```
0 → 1 → 2 → 3 → ... → TOP
↑                       │
└───────────────────────┘
```

Il compte uniquement vers le haut.

Par exemple avec :

TOP = 9

```
OCR1A = 3
```

on peut avoir :

```
TCNT1 : 0 1 2 3 4 5 6 7 8 9
```

OC1A : 1 1 1 0 0 0 0 0 0 0
Puis ça recommence.

Fast PWM est pratique quand on veut générer un PWM avec une fréquence relativement élevée.

Modes Fast PWM 8, 9 et 10 bits

Timer1 peut utiliser des valeurs maximales fixes.

Mode 5 — Fast PWM 8 bits

WGM = 0101
TOP = 255
Le Timer compte :

```
0 → 255
```

puis repart à zéro.

Pourquoi 255 ?

Parce que :

```
8 bits → 2^8 = 256 valeurs
```

donc :

0 à 255

Mode 6 — Fast PWM 9 bits

TOP = 511
Parce que :

2^9 = 512 valeurs
donc :

```
0 → 511
```

Mode 7 — Fast PWM 10 bits

TOP = 1023
Parce que :

2^10 = 1024 valeurs
donc :

```
0 → 1023
```

Mode 14 — Fast PWM avec ICR1

Celui-ci est particulièrement intéressant.

WGM = 1110
Ici :

```
TOP = ICR1
```

Donc c**'est toi qui choisis la valeur maximale.**
Par exemple :

```
ICR1 = 62499;
```

Timer1 compte :

```
0 → 62499
```

puis revient à zéro.

C'est très flexible.

Et OCR1A peut servir à définir le rapport cyclique.

Donc tu peux voir :

```
ICR1  → durée totale de la période
OCR1A → moment où la sortie change
```

C’est pour ça que ce mode est intéressant pour ton exercice.

Mode 15 — Fast PWM avec OCR1A

WGM = 1111
Ici :

```
TOP = OCR1A
```

C’est similaire, sauf qu’on utilise OCR1A pour fixer le maximum.

Le problème est qu’ensuite OCR1A est déjà utilisé comme TOP.

Donc si tu veux utiliser le canal A pour contrôler précisément le rapport cyclique, ça peut être moins pratique.

C’est une des raisons pour lesquelles on préfère souvent :

```
TOP = ICR1
```

pour ton type d’exercice.

Phase Correct PWM

Ici, le fonctionnement change.

Fast PWM faisait :

```
0 → TOP → 0 → TOP → 0
```

Le P**hase Correct PWM** fait :

```
0 → TOP → 0
```

mais en comptant réellement vers le haut puis vers le bas.

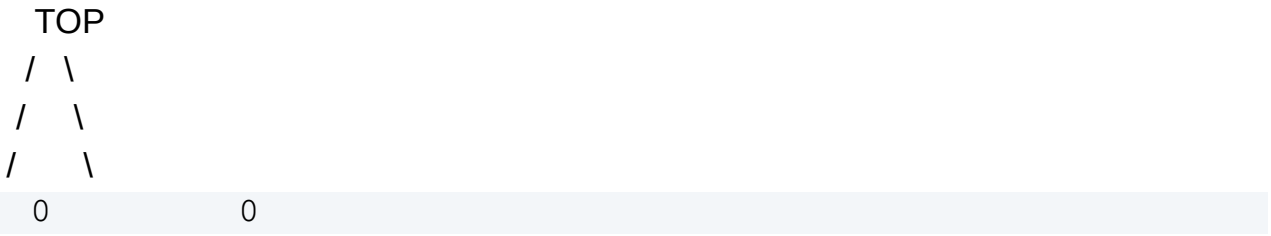
Par exemple :

```
0
1
```

2
3
4
5
4
3
2

```
1
0
1
```

2
...
Donc le compteur fait :



On appelle ça parfois d**ual slope** : double pente.

Fast PWM est plutôt :

```
0 → TOP
0 → TOP
0 → TOP
```

alors que Phase Correct est :

```
0 → TOP → 0 → TOP → 0
```

L’intérêt est d’avoir un signal PWM plus symétrique autour de son centre.

C’est utile notamment dans certaines commandes de moteurs et certains systèmes où la symétrie du signal compte.

Mode 1 — Phase Correct PWM 8 bits

TOP = 255
Le Timer fait :

```
0 → 255 → 0
```

Mode 2 — Phase Correct PWM 9 bits

TOP = 511

Mode 3 — Phase Correct PWM 10 bits

TOP = 1023

Mode 10 — Phase Correct PWM avec ICR1

Ici :

```
TOP = ICR1
```

Donc tu choisis la valeur maximale.

Le compteur fait :

```
0 → ICR1 → 0
```

Mode 11 — Phase Correct PWM avec OCR1A

Ici :

```
TOP = OCR1A
```

et le compteur fait :

```
0 → OCR1A → 0
```

Phase and Frequency Correct PWM

Il ressemble beaucoup au Phase Correct PWM.

Le Timer fait toujours :

```
0 → TOP → 0
```

La différence est surtout q**uand les nouvelles valeurs sont prises en compte** quand tu modifies TOP.
C’est une différence plus technique.

> **Pour l’instant, retiens :**

Phase Correct
`→ cherche principalement une PWM symétrique`

Phase and Frequency Correct

```
→ PWM symétrique + meilleure gestion des changements de fréquence
```

C’est surtout intéressant lorsqu’on modifie la fréquence pendant que le Timer fonctionne.

Mode 8 — Phase and Frequency Correct avec ICR1

```
TOP = ICR1
```

Le Timer fait :

```
0 → ICR1 → 0
```

Mode 9 — Phase and Frequency Correct avec OCR1A

```
TOP = OCR1A
```

Même principe :

```
0 → OCR1A → 0
```

Tableau complet de Timer1

Tu peux garder celui-là comme référence :

OBJ

Mode	WGM	Type	TOP
0	0	Normal	0xFFFF = 65535
1	1	PWM Phase Correct 8 bits	0x00FF = 255
2	10	PWM Phase Correct 9 bits	0x01FF = 511
3	11	PWM Phase Correct 10 bits	0x03FF = 1023
4	100	CTC	OCR1A
5	101	Fast PWM 8 bits	255
6	110	Fast PWM 9 bits	511
7	111	Fast PWM 10 bits	1023
8	1000	Phase & Frequency Correct PWM	ICR1
9	1001	Phase & Frequency Correct PWM	OCR1A
10	1010	Phase Correct PWM	ICR1
11	1011	Phase Correct PWM	OCR1A
12	1100	CTC	ICR1

13	1101	réservé	—
14	1110	Fast PWM	ICR1
15	1111	Fast PWM	OCR1A

Mais surtout, n**’essaie pas d’apprendre les 16 modes par cœur.**

Retient plutôt les quatre grandes idées :

NORMAL

```
→ compte jusqu'au maximum naturel puis déborde
```

CTC

```
→ compte jusqu'à une valeur choisie puis repart à 0
```

FAST PWM

```
→ compte vers le haut jusqu'à TOP puis revient directement à 0
```

PHASE CORRECT

```
→ compte vers le haut jusqu'à TOP puis redescend jusqu'à 0
```

Et dans t**on exercice 1 Hz + 10 %,** le mode qui nous intéresse particulièrement est :

Mode 14

Fast PWM

```
TOP = ICR1
```

15. TOP, ICR1 et OCR1A

Alors on fait juste T**OP** et pourquoi ICR1/OCR1A.
TOP, c'est simplement l**a valeur maximale jusqu'à laquelle le timer compte avant de recommencer.**

Exemple très simple :

TOP = 9
Alors le timer fait :

0 1 2 3 4 5 6 7 8 9
puis il repart à 0.

Donc TOP fixe en gros l**a longueur totale d'une période.**

Dans notre exercice, on veut une période de :

1 seconde
Donc on veut choisir un TOP qui fasse durer le comptage exactement 1 seconde.

Maintenant, pourquoi utiliser ICR1 ?

Parce qu'en mode 14, on dit au Timer1 :

```
TOP = ICR1
```

Donc `ICR1` sert à définir :

jusqu'où le timer compte avant de recommencer.

Ensuite, `OCR1A` sert à dire :

à quel moment, pendant cette période, la sortie OC1A change d'état.

Donc tu peux voir ça comme :

```
ICR1  = fin de la période
OCR1A = moment où la LED s'éteint
```

Exemple imaginaire :

```
ICR1 = 100
OCR1A = 10
```

Le timer compte :

0 ---------- 10 ---------- 100
| | |

début comparaison fin

Et si on configure la sortie pour être haute au début puis basse après `OCR1A` :

```
0 à 10     → LED ON
10 à 100   → LED OFF
```

Donc :

10 sur 100 = 10 %
C’est exactement ce qu’on veut.

> **Donc mentalement :**

```
ICR1 = 100 % de la période
OCR1A = portion ON
```

Dans ton vrai exercice, ce ne sera pas 100 et 10, mais l’idée est exactement la même.

Le gros avantage de séparer les deux est que tu peux changer le rapport cyclique sans changer la fréquence.

Par exemple :

```
ICR1 reste pareil
```

mais tu peux faire :

```
OCR1A = 10 %  → LED ON 10 %
OCR1A = 50 %  → LED ON 50 %
OCR1A = 80 %  → LED ON 80 %
```

Donc :

```
ICR1 contrôle la fréquence
OCR1A contrôle le rapport cyclique
```

C’est vraiment la phrase la plus importante à retenir pour ce montage.

16. COM1A en Fast PWM : mode inversé et non-inversé

Oui. Là, COM1A1 et COM1A0 servent toujours à décider c**e que Timer1 fait sur OC1A,** mais en PWM leur rôle est un peu différent du mode CTC.

Dans ton ancien exercice en CTC, on utilisait :

```
COM1A1 = 0
COM1A0 = 1
```

pour dire :

à chaque comparaison, inverse OC1A.

Donc :

```
0 → 1 → 0 → 1 → ...
```

En F**ast PWM,** on ne veut plus juste inverser la sortie. On veut fabriquer un signal avec une partie ON et une partie OFF.

Pour notre exercice, on choisit :

```
COM1A1 = 1
COM1A0 = 0
```

Cette combinaison correspond au mode n**on-inversé.**

Ça veut dire :

au début de chaque période, OC1A est mise à 1 ; quand TCNT1 atteint OCR1A, OC1A passe à 0.

Prenons un exemple simple :

```
ICR1  = 100
OCR1A = 10
```

Le compteur fait :

```
TCNT1 :
```

0 ---------- 10 ---------- 100

```
↑                    ↑                                   ↑
```

début comparaison TOP

Et la sortie fait :

OC1A :

1 1 1 1 1 1 1 1 1 1 | 0 0 0 0 0 ... 0

```
                        ↑
                TCNT1 = OCR1A
```

Donc :

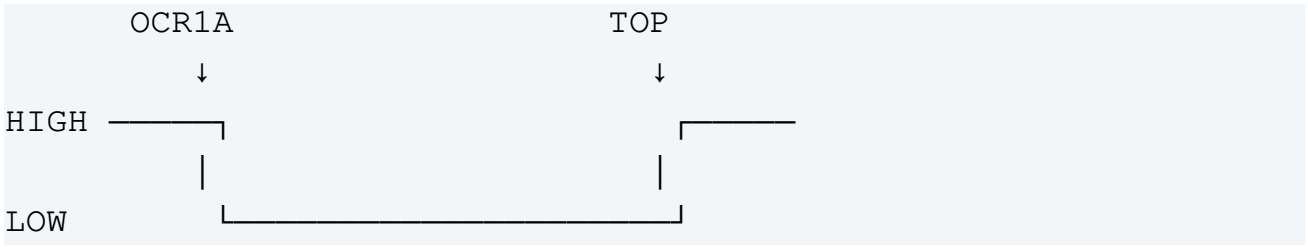
```
de 0 à 10     → HIGH
de 10 à 100   → LOW
```

Puis, lorsque le compteur atteint TOP, une nouvelle période commence :

```
TCNT1 revient à 0
```

OC1A revient à 1
Et ça recommence.

Donc visuellement :



10 % 90 %

C'est exactement ce qu'on veut pour notre LED.

Pourquoi ça s'appelle « non-inversé » ?

Parce que plus OCR1A est grand, plus longtemps la sortie reste à 1.

Par exemple :

```
OCR1A = 10 % du TOP
→ 10 % HIGH
OCR1A = 50 % du TOP
→ 50 % HIGH
OCR1A = 80 % du TOP
→ 80 % HIGH
```

C'est assez intuitif.

Il existe aussi le mode i**nversé** :

```
COM1A1 = 1
COM1A0 = 1
```

Là, c'est l'inverse :

```
au début → OC1A = 0
à OCR1A  → OC1A = 1
```

Donc avec OCR1A = 10 % :

LOW pendant 10 %

HIGH pendant 90 %
Ce n'est pas ce qu'on veut ici.

Tu peux donc retenir pour le Fast PWM :

```
COM1A1 COM1A0
  0      0   → OC1A non contrôlée par le Timer
  1      0   → PWM non-inversé
  1      1   → PWM inversé
```

Le cas 01 est un cas spécial selon le mode PWM utilisé ; pour notre m**ode 14,** on ne s'en sert pas.

Donc pour ton exercice :

```
TCCR1A = (1 << COM1A1) | ...;
```

met :

```
COM1A1 = 1
COM1A0 = 0
```

et veut dire :

mettre PB1/OC1A à HIGH au début de la période puis à LOW quand le compteur atteint OCR1A.

C'est cette mécanique qui permet de fabriquer précisément les 1**0 % ON / 90 % OFF.**